Rakesh Nikumbh

SPRING FRAMEWORK & SPRING BOOT - COMPLETE SYLLABUS

1. SPRING CORE (IoC & DI)

- Introduction to Spring Framework
- Benefits of Spring
- IoC Container & DI
- Beans and Bean Lifecycle
- Bean Scopes (Singleton, Prototype, Request, Session, Application)
- Dependency Injection Types (Constructor, Setter, Field)
- Autowiring
- Qualifiers and Primary
- XML based Configuration
- Annotation based Configuration
- Java based Configuration

2. SPRING CONTEXT

- ApplicationContext vs BeanFactory
- Types of ApplicationContext (ClassPathXml, FileSystemXml, AnnotationConfig, WebApplication)
- Loading Properties File
- Profile and Environment
- MessageSource (i18n)
- Event Handling in Spring
- Resource Handling

3. SPRING DATA JPA

- Introduction to JPA & Hibernate
- Persistence Context
- Entity Mapping (@Entity, @Table, @Id, @Column, @OneToMany, @ManyToOne)
- CRUD Operations
- Repository Interface
- Query Methods
- @Query (JPQL, Native Query)
- Pagination & Sorting
- Transactions (@Transactional)
- Auditing
- Entity Lifecycle

4. SPRING MVC

- MVC Architecture Overview
- DispatcherServlet
- Controllers (@Controller, @RequestMapping)
- Request Mapping (GET, POST, PUT, DELETE)
- Request Parameters & Path Variables
- Model and ModelMap
- View Resolvers (InternalResourceViewResolver)
- Form Handling (@ModelAttribute)
- Data Binding & Validation
- Exception Handling (@ExceptionHandler, @ControllerAdvice)
- Interceptors
- File Upload and Download
- Static Resources Handling

5. SPRING AOP

- Introduction to AOP
- Need of AOP
- Core AOP Concepts (Aspect, JoinPoint, Advice, Pointcut, Weaving)
- Types of Advice (Before, After, Around, AfterReturning, AfterThrowing)
- Pointcut Expressions
- AOP with Annotations (@Aspect, @Before, etc.)
- XML based AOP Configuration
- Use Cases (Logging, Security, Transactions)

6. SPRING REST

- REST Principles
- @RestController vs @Controller
- @RequestMapping (GET, POST, PUT, DELETE)
- @PathVariable, @RequestParam
- @RequestBody, @ResponseBody
- ResponseEntity
- Exception Handling for REST APIs
- Validation
- RESTful Best Practices
- HATEOAS (Basics)

7. SPRING SECURITY

- Introduction to Spring Security
- Authentication vs Authorization
- Architecture of Spring Security
- Setting up Spring Security
- In-Memory Authentication
- JDBC Authentication
- Custom UserDetailsService
- Password Encoding (BCrypt)
- Role Based Authorization
- Method Level Security (@PreAuthorize)
- CSRF Protection
- Form Login, Logout
- Remember Me
- Basic Authentication
- Security with REST APIs

8. SPRING BOOT

- Introduction to Spring Boot
- Features and Benefits
- Spring Boot Starters
- Auto Configuration
- Application Properties
- Profiles (dev, test, prod)
- Spring Boot CLI
- Actuator (Monitoring & Management)
- Embedded Servers (Tomcat, Jetty)
- Building Executable JAR/WAR
- Spring Initializr

9. SPRING MICRO-SERVICES

- Introduction to Microservices
- Advantages & Challenges
- Microservices Architecture
- Service Discovery (Eureka Server & Client)
- API Gateway (Spring Cloud Gateway)
- Configuration Server
- Load Balancing (Ribbon)
- Circuit Breaker (Resilience4j)
- Distributed Tracing (Sleuth + Zipkin)
- Centralized Logging

10. SPRING JWT (JSON WEB TOKEN)

- Introduction to JWT
- Structure of JWT
- Advantages of JWT
- Generating JWT Token
- Validating JWT Token
- Stateless Authentication with JWT
- Refresh Token
- JWT with Spring Security

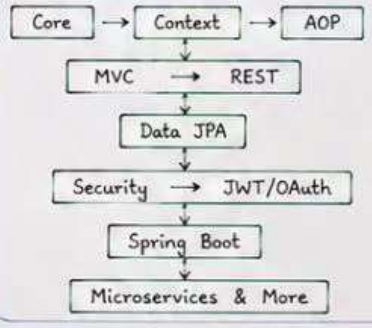
11. SPRING OAUTH 2.0

- Introduction to OAuth 2.0
- Roles in OAuth 2.0 (Resource Owner, Client, Authorization Server, Resource Server)
  OAuth 2.0 Grant Types (Authorization Code, Password, Client Credentials, Refresh Token)
- Implementing OAuth 2.0
- Social Login (Google, Facebook)
- OAuth 2.0 with Spring Security

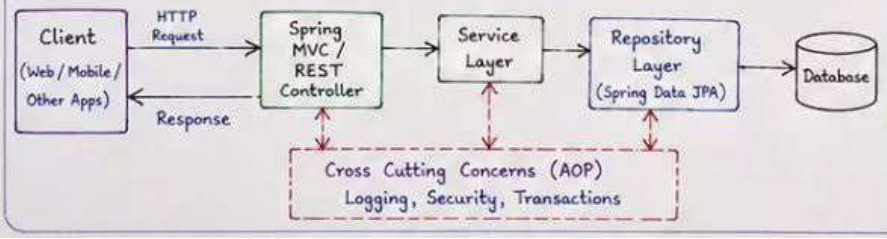
12. MORE IMPORTANT TOPICS

- Spring Validation (@Valid)
- Internationalization (i18n)
- Caching (Spring Cache Abstraction)
- Scheduling (@Scheduled)
- Spring Mail
- WebSocket Support
- Testing in Spring (JUnit, Mockito)
- Logging (SLF4J, Logback)
- Handling Multiple Environments
- Best Practices in Spring

TYPICAL LEARNING FLOW



SPRING APPLICATION ARCHITECTURE (TYPICAL)



TOOLS & TECHNOLOGIES

- Spring Tool Suite (STS) / IntelliJ IDEA
- Maven / Gradle
- MySQL / PostgreSQL / Oracle
- Postman
- Git & GitHub
- Docker (for Microservices)
- Eureka, Zipkin, Prometheus (Monitoring)

Master Spring Ecosystem and build robust, scalable and secure enterprise applications!

SPRING CORE – COMPLETE SYLLABUS NOTES

Rakesh Nikumbh

Main Concepts Covered

1. Introduction to Spring Framework

- What is Spring?
- Spring is a lightweight framework for Java application development.
- It provides infrastructure support for building Java applications.
- Core features – IoC, DI, AOP, Container, Transaction, MVC, etc.
- Benefits – Loose coupling, Testability, Modularity, Maintainability.

2. Benefits of Spring

- Lightweight and modular
- Loose coupling
- Easy unit testing
- AOP support
- Declarative transactions
- JDBC exception handling
- Integration with other frameworks

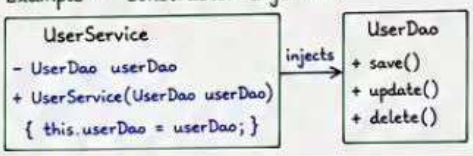
3. IoC (Inversion of Control)

- IoC is a design principle where the control of object creation and management is transferred from the application to the Spring container.
- Spring IoC container is responsible for creating, configuring and managing objects.

4. DI (Dependency Injection)

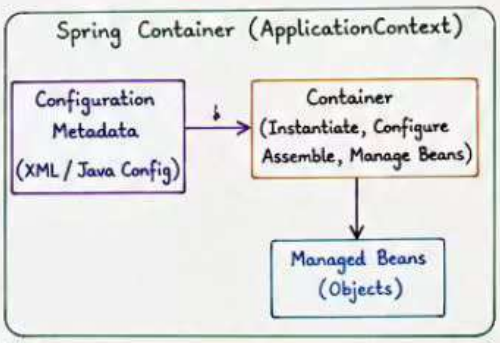
- DI is a technique to achieve IoC.
- Objects get their dependencies from external source (Spring container) instead of creating them by themselves.
- Types of DI :
  1. Constructor Injection
  2. Setter Injection
  3. Field Injection

Example – Constructor Injection

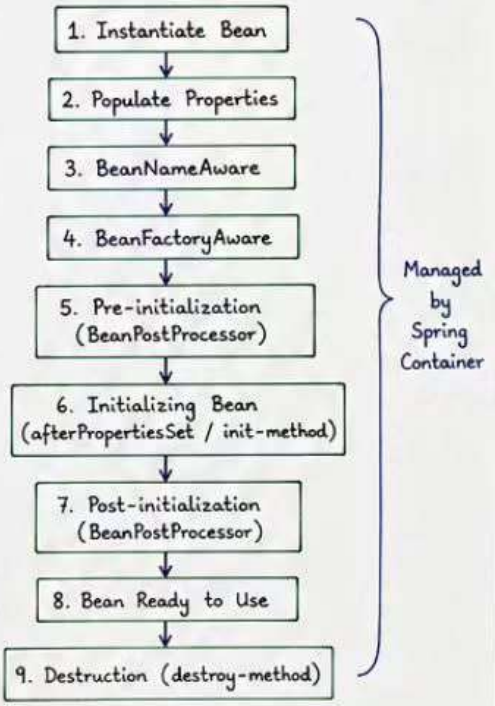


5. Spring Container

- The heart of Spring Framework.
- Creates, configures and manages beans.
- Types of Container :
  1. BeanFactory (Basic container)
  2. ApplicationContext (Advanced container)



7. Bean Lifecycle



6. Beans in Spring

- Bean is an object managed by Spring IoC container.
- Defined in XML or Java Config.
- Bean Lifecycle states :
  1. Instantiation
  2. Population of properties
  3. BeanNameAware
  4. BeanFactoryAware
  5. Pre-initialization (BeanPostProcessor)
  6. Initializing Bean (afterPropertiesSet or init-method)
  7. Post-initialization (BeanPostProcessor)
  8. Bean is ready to use
  9. Destruction (destroy-method)
- Scopes of Bean :
  1. singleton (default)
  2. prototype
  3. request
  4. session
  5. application
  6. websocket

8. Configuration Metadata

Ways to configure Spring :

① XML Configuration

```
<bean id="userDao" class="com.app.UserDao" />
```

② Java Based Configuration

```
@Configuration
public class AppConfig {
  @Bean
  public UserDao userDao() {
    return new UserDaoImpl();
  }
}
```

③ Annotation Based Configuration

Using @Component, @Service, @Repository, @Configuration, @Bean, etc.

9. Autowiring

- Spring can automatically inject dependencies.
- Modes :
  1. byName
  2. byType
  3. constructor
  4. autodetect (default)
  5. @Qualifier (to resolve ambiguity)

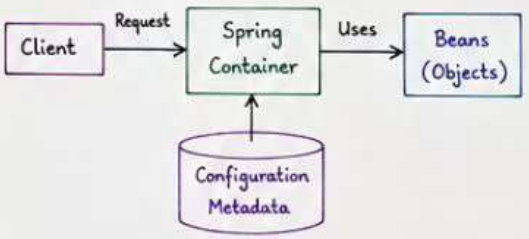
10. Annotations in Spring Core

- @Component – Generic component
- @Service – Service layer
- @Repository – DAO layer
- @Controller – Web layer (in Spring MVC)
- @Configuration – Java config class
- @Bean – Bean definition method

11. Spring Core Other Topics

- Spring Expression Language (SpEL)
  - Used in configuration and annotations.
  - Example: #{1 + 2}, #{user.name}
- Property Placeholder
  - Externalize configuration using .properties file.
  - ${property.name}
- Type Conversion
  - Automatic type conversion by Spring.
  - Uses Converter.
- Resource Handling
  - Supports different resource types.
  - ClassPathResource, FileSystemResource, etc.
- Spring Events

12. How Spring Works (Flow)



Summary

- ✓ Spring Core provides the foundation.
- ✓ IoC & DI are the core principles.

✳ SPRING CONTEXT – COMPLETE SYLLABUS NOTES ✳

Rakesh Nikumbh

Definition : Spring Context is the interface that represents the Spring IoC container. It builds on top of the BeanFactory and adds enterprise-specific features.

1. ApplicationContext vs BeanFactory

BeanFactory	ApplicationContext (Spring Context)
• Basic IoC container.	• Advanced container with enterprise features.
• Lazy loading by default.	• Eager loading (singleton beans created at startup).
• No support for i18n, events, AOP, etc.	• Supports i18n, event handling, AOP, resource handling, etc.
• Minimal features.	• Full-fledged framework features.
• Used in small applications.	• Used in real-time, enterprise applications.

2. Types of ApplicationContext

- ClassPathXmlApplicationContext
  Loads configuration file from classpath.
- FileSystemXmlApplicationContext
  Loads configuration file from the file system.
- AnnotationConfigApplicationContext
  Uses Java configuration (@Configuration classes) instead of XML.
- WebApplicationContext (used in web applications)
  - XmlWebApplicationContext
  - AnnotationConfigWebApplicationContext

3. Loading Properties File

- Spring Context can load .properties file using PropertyPlaceholderConfigurer (XML) or @PropertySource (Java Config).

Example (XML) :

```
<bean id="propertyConfigurer"
   class="org.springframmerk.beans.factory.config.
   PropertyPlaceholderConfigurer">
   <property name="location" value="classpath:app.properties"/>
</bean>
```

4. Profile and Environment

- Used to create different configurations for different environments (dev, test, prod).
- @Profile("dev") can be used on beans.
- Environment object provides access to properties and active profiles.

5. MessageSource (i18n)

- Supports internationalization (i18n).
- Uses .properties files based on locale.
- MessageSource interface is used.
- Example files : messages_en.properties, messages_hi.properties

Spring Context Architecture

Configuration Metadata (XML / Java Config / Annotations)
↓
ApplicationContext (Interface)
↓
BeanFactory (creates & manages beans)
↓
Beans / Objects (Managed by Spring)

6. Event Handling in Spring

- Spring provides event handling mechanism.
- Components :
  - ApplicationEvent
  - ApplicationListener
  - ApplicationEventPublisher
- Flow :

Event Publisher —publishes→ Application Event —handled by→ Event Listener

Example :

```
public class MyEvent extends ApplicationEvent {
   public MyEvent(Object source) {
      super(source);
   }
}

@Component
public class MyListener implements
   ApplicationListener<MyEvent> {
   public void onApplicationEvent(MyEvent event) {
      // handle event
   }
}
```

7. Resource Handling

- Spring provides Resource interface to abstract access to resources.
- Supports classpath, file system, URL, etc.
- Example :

```
Resource resource =
      new ClassPathResource("data.txt");
InputStream is = resource.getInputStream();
```

8. Other Important Features

- Easier integration with other frameworks.
- Life-cycle management of beans.
- Supports AOP, transactions, security, etc.
- Refreshable ApplicationContext.
- Hierarchical contexts (parent-child).

9. Bean Definition in Context

- Beans can be defined using :
  - XML configuration
  - Annotations
  - Java Config
- Scope of beans : singleton (default), prototype, request, session, global-session.
- Lifecycle methods :
  - init-method
  - destroy-method

10. Annotations in Spring Context

- @ComponentScan – for component scanning
- @Component – generic stereotype
- @Service – for service layer
- @Repository – for DAO layer
- @Controller – for web layer
- @Configuration – Java config class
- @Bean – bean definition method
- @Import – import other config class
- @PropertySource – load property file
- @Profile – environment based config

11. Advantages

- Better than BeanFactory.
- Easy integration with enterprise features.
- Supports resource management.
- Supports i18n, events, profiles.
- Better for real-time applications.

12. Example – XML Based Context

```
<beans xmlns="http://www.springframmeork.org/
   schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://www.springframmeork.org/
   schema/beans http://www.springframmeork.org/
   schema/beans/spring-beans.xsd">

   <bean id="userService" class="com.app.UserService"/>
   <bean id="userDao" class="com.app.UserDaoImpl"/>
</beans>
```

13. Example – Java Config Based Context

```
@Configuration
@ComponentScan(basePackages = "com.app")
public class AppConfig {
   @Bean
   public UserService userService() {
      return new UserServiceImpl(userDao());
   }
   @Bean
   public UserDao userDao() {
      return new UserDaoImpl();
   }
}
```

SUMMARY – SPRING CONTEXT

SPRING DATA JPA – COMPLETE SYLLABUS NOTES

Rakesh Nikumbh

Definition : Spring Data JPA is a part of Spring Data project that makes it easy to implement JPA based repositories. It reduces boilerplate code and provides powerful abstractions.

Benefits
- ✓ Reduces boilerplate code
- ✓ Easy CRUD operations
- ✓ Pagination & Sorting support
- ✓ Custom query methods
- ✓ Integration with Spring Boot
- ✓ Powerful & flexible

1. Introduction to Spring Data JPA

- Part of Spring Data project.
- Builds on top of JPA (Hibernate).
- Provides Repository abstraction.
- Automatic implementation of repository interfaces at runtime.
- Follows Convention over Configuration.

2. JPA Basics (Recap)

- JPA is a specification for ORM.
- Implemented by Hibernate in Spring Boot.
- Uses @Entity, @Table, @Id, etc.
- Entity lifecycle: Transient, Persistent, Detached, Removed.

3. Dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

4. Entity Mapping

- @Entity – marks class as entity
- @Table(name = "table_name") – table mapping
- @Id – primary key
- @GeneratedValue – auto generation
- @Column – column mapping
- Relationships mapping:
  - @OneToOne, @OneToMany, @ManyToOne, @ManyToMany
- Cascade operations
- Fetch types: LAZY (default), EAGER

Example :

```
@Entity
@Table(name = "students")
public class Student {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @Column(name = "name", nullable = false)
  private String name;
  private String email;
}
```

5. Repository Interface

- Repository interfaces extend JpaRepository (or CrudRepository).
- Provides CRUD methods out of the box.

```
public interface StudentRepository
  extends JpaRepository<Student, Long> {
  // No implementation required
}
```

CRUD Methods Provided
- save(S entity)
- findById(ID id)
- findAll()
- count()
- deleteById(ID id)
- delete(S entity)

6. Derived Query Methods

- Create query methods by following naming conventions.

Examples :

```
List<Student> findByName(String name);
List<Student> findByEmail(String email);
List<Student> findByNameContaining(String name);
List<Student> findByNameAndEmail(String name, String email);
List<Student> findByNameOrderByIdDesc(String name);
```

7. @Query Annotation

- Used for custom JPQL or Native SQL queries.

Example : JPQL

```
@Query("select s from Student s where s.name = :name")
List<Student> findByNameCustom(@Param("name") String name);
```

Example : Native SQL

```
@Query(value = "select * from students where email = ?1",
       nativeQuery = true)
Student findByEmailNative(String email);
```

8. Pagination and Sorting

- Supports Pagination using Page and Pageable.
- Supports Sorting using Sort.

Example :

```
Pageable pageable = PageRequest.of(0, 5, Sort.by("name").ascending());
Page<Student> page = studentRepository.findAll(pageable);

Sort sort = Sort.by(Sort.Direction.DESC, "id");
List<Student> list = studentRepository.findAll(sort);
```

9. Projections

- Used to fetch specific fields only.

1) Interface Based Projection

```
public interface StudentView {
  String getName();
  String getEmail();
}

@Query("select s from Student s")
List<StudentView> findAllStudents();
```

2) DTO Based Projection

```
@Query("select new com.example.dto.StudentDto(s.id,
        s.name, s.email) from Student s")
List<StudentDto> findAllDto();
```

10. Modifying Queries

- For UPDATE or DELETE queries.
- Use @Modifying and @Transactional.

Example :

```
@Modifying
@Transactional
@Query("update Student s set s.email = ?2 where s.id = ?1")
int updateEmail(Long id, String email);
```

11. Transactions

- Spring Data JPA works with @Transactional.
- Read operations are transactional by default in Spring Boot.

12. Auditing (Spring Data JPA Auditing)

- Automatically fill createdBy, createdDate, lastModifiedBy, lastModifiedDate.

Steps :
1. Add dependency : spring-boot-starter-data-jpa
2. Enable auditing : @EnableJpaAuditing
3. Extend entity from auditor classes or use annotations like @CreatedDate, @LastModifiedDate

13. Specifications (Dynamic Queries)

- Build type-safe, dynamic queries using JPA Criteria API.

Example :

```
Specification<Student> spec = (root, query, cb) ->
  cb.equal(root.get("name"), "Aadi");
List<Student> students = studentRepository.findAll(spec);
```

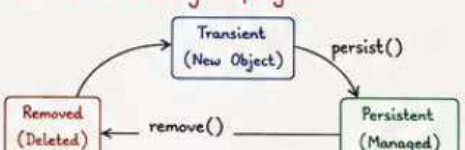
14. Query by Example (QBE)

- Query using Example and ExampleMatcher.
- Useful for simple dynamic queries.

Example :

```
Student probe = new Student();
probe.setName("Aadi");
Example<Student> example = Example.of(probe);
List<Student> students = studentRepository.findAll(example);
```

15. JPA Entity Lifecycle



16. Best Practices

- Use meaningful method names.
- Use DTO projections for performance.
- Avoid N+1 select problem (use fetch join).
- Use pagination for large data.

SPRING MVC – COMPLETE SYLLABUS NOTES

Rakesh Nikumbh

Definition : Spring MVC is a part of Spring Framework. It is a web application framework based on Model-View-Controller design pattern. It helps to build web applications with clean separation of concerns and provides flexible configuration.

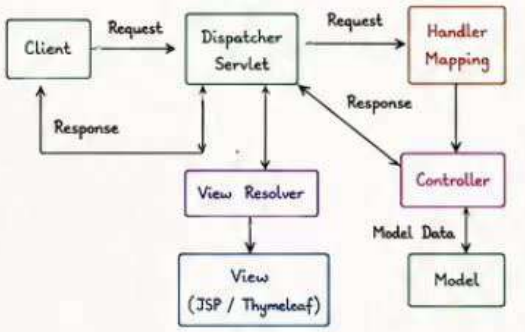
Benefits

- ✓ Clean separation of concerns
- ✓ Flexible and loosely coupled
- ✓ Easy integration with other frameworks
- ✓ Support for RESTful web services
- ✓ Testable and maintainable code
- ✓ Powerful data binding and validation.

1. Overview of Spring MVC

- Built on top of Spring Core.
- Based on Model-View-Controller pattern.
- Front Controller: DispatcherServlet.
- Configured using Java Config / XML.
- Supports MVC, REST, Validation, i18n, etc.

2. Architecture of Spring MVC



Flow : Client -> DispatcherServlet -> HandlerMapping -> Controller -> Model -> ViewResolver -> View -> Response

3. DispatcherServlet (Front Controller)

- Central servlet that handles all requests.
- Receives request and delegates to appropriate controller.
- Configured in web.xml or via @WebServlet.

```
web.xml configuration
<servlet>
 <servlet-name>dispatcher</servlet-name>
 <servlet-class>
  org.springframmoct.web.servlet. DispatcherServlet
 </servlet-class>
 <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
 <servlet-name>dispatcher</servlet-name>
 <url-pattern>/</url-pattern>
</servlet-mapping>
```

4. Handler Mapping

- Maps incoming request to the appropriate controller.
- Default: RequestMappingHandlerMapping.
- Handles URL patterns and HTTP methods.

5. Controllers

- Handle user requests and return ModelAndView.
- Annotated with @Controller or @RestController.
- Use @RequestMapping to map URLs.

Example :

```
@Controller
public class HomeController {
  @RequestMapping("/hello")
  public String hello(Model model) {
    model.addAttribute("msg", "Hello Spring MVC");
    return "hello"; // view name
  }
}
```

6. Request Mapping

- Used to map web requests to handler methods.
- @RequestMapping is versatile.
- Shortcuts :
  - @GetMapping – for GET requests
  - @PostMapping – for POST requests
  - @PutMapping – for PUT requests
  - @DeleteMapping – for DELETE requests
  - @PatchMapping – for PATCH requests

7. Method Arguments

- Spring MVC can bind request data to method arguments automatically.
- Supported arguments :
  - HttpServletRequest, HttpServletResponse
  - @RequestParam
  - @PathVariable
  - @RequestHeader
  - @CookieValue
  - @ModelAttribute
  - RequestBody (for REST)
  - Model, Map

8. Return Types

- String – view name
- ModelAndView – model + view
- void – response handled
- @ResponseBody / @RestController – return data (JSON, XML)

9. Model and ModelAndView

- Model is used to hold data for the view.
- Data is exposed to the view.

Example :

```
Model model = new ModelAndMap();
model.addAttribute("user", user);
```

10. View Resolver

- Resolves logical view names to actual views.
- Common ViewResolvers :
  - InternalResourceViewResolver (for JSP)
  - ThymeleafViewResolver
- Configured prefix and suffix.

Example :

```
bean.setPrefix("/WEB-INF/views/");
bean.setSuffix(".jsp");
```

11. Views

- Responsible for rendering the UI.
- Technologies :
  - JSP
  - Thymeleaf
  - FreeMarker
  - Velocity
- Keep views free from Java code (use EL, JSTL).

12. Data Binding

- Binds request parameters to Java objects.
- Uses @ModelAttribute.

Example :

```
@PostMapping("/save")
public String save(@ModelAttribute User user) {
   // user object is populated
   return "success";
}
```

13. Validation

- Use JSR-303 (Bean Validation).
- Annotations like @NotNull, @Size, @Email etc.
- Use @Valid to trigger validation.

Example :

```
public String save(@Valid @ModelAttribute User user,
                   BindingResult result) {
  if(result.hasErrors())
     return "form";
  return "success";
}
```

14. Exception Handling

- Handle exceptions gracefully.
- Using @ExceptionHandler, @ControllerAdvice.

Example :

```
@ControllerAdvice
public class GlobalExceptionHandler {
 @ExceptionHandler(Exception.class)
  public String handle(Exception ex, Model model){
    model.addAttribute("error", ex.getMessage());
    return "error";
  }
}
```

15. File Upload

- Use MultipartResolver.
- Handle file upload using MultipartFile.

Example :

```
@PostMapping("/upload")
public String upload(@RequestParam MultipartFile file){
   // handle file
   return "success";
}
```

16. Interceptors

- Used for pre/post processing of requests.
- Implement HandlerInterceptor.
- Common use cases:

17. Internationalization (i18n)

- Supports multiple languages.
- Use ResourceBundleMessageSource.
- Message files: messages_en.properties, messages_hi_properties
- Use <spring:message code="msg.key" /> in JSP or #{msg.key} in Thymeleaf.

19. Other Important Topics

- Static Resources Handling (CSS, JS, Images)
- Flash Attributes
- Redirect vs Forward
- Form Handling
- PRG Pattern (Post/Redirect/Get)
- Content Negotiation (JSON, XML)
- Testing Spring MVC (MockMvc)

SPRING AOP - COMPLETE SYLLABUS NOTES

Rakesh Nikumbh

Definition : Spring AOP (Aspect Oriented Programming) provides a way to modularize cross-cutting concerns such as logging, security, transaction management, caching etc. It helps to keep the core business logic clean and focused.

Benefits

- ✓ Separation of cross-cutting concerns
- ✓ Improves code reusability and maintainability
- ✓ Reduces code duplication
- ✓ Easier to manage and modify
- ✓ Supports declarative transactions, security etc.

1. Introduction to AOP

- AOP is an extension of OOP.
- Focuses on cross-cutting concerns.
- Implemented in Spring using proxy-based model.
- Configured using XML or @AspectJ annotations.

2. Key Concepts in AOP

- Aspect – A module that contains advice.
- Join Point – A point in the execution of program (method execution in Spring AOP).
- Advice – Action taken at a join point.
- Pointcut – Expression to match join points.
- Weaving – Linking aspects with target objects to create a advised object.
- Introduction – Adding new behavior to existing classes.
- Target Object – Object which is being advised.
- Proxy – Object created by Spring AOP.

3. Advice Types

- @Before – Executed before the method.
- @AfterReturning – After method successfully completes.
- @AfterThrowing – After method throws an exception.
- @After (finally) – After method execution (success/exception).
- @Around – Before and after method execution.

4. Pointcut

- Expression to select join points.
- Uses AspectJ pointcut expression language.
- Examples :

```
execution(* com.example.service.*.*(..))  – all methods within service package
execution(* com.example.service.*.save(..)) – methods named save
within(com.example.service.*)             – within package
@annotation(com.example.Log)              – methods with @Log annotation
args(String)                              – methods with String argument
```

5. Join Point

- In Spring AOP, join point always represents method execution.
- Can be method call, exception handling etc. in AspectJ (not in Spring AOP).

6. Aspect

- A class that contains advice and pointcut.
- Annotated with @Aspect.
- Contains methods annotated with advice annotations.
- Example :

```
@Aspect
public class LoggingAspect {
   @Before("execution(* com.example.service.*.*(..))")
   public void logBefore(JoinPoint jp) {
      System.out.println("Before Method: " + jp.getSignature());
   }
}
```

7. Weaving

8. XML Based Configuration

- Enable AOP namespace.
- Configure aspect and pointcut in XML.
- Example :

```
<aop:config>
   <aop:aspect ref="loggingAspect">
      <aop:pointcut id="allMethods"
         expression="execution(* com.example.service.*.*(..))"/>
      <aop:before pointcut-ref="allMethods"
         method="logBefore" />
   </aop:aspect>
</aop:config>
```

9. Annotation Based Configuration

- Use @EnableAspectJAutoProxy to enable AOP.
- Define aspect using @Aspect.
- Define pointcut using @Pointcut.
- Example :

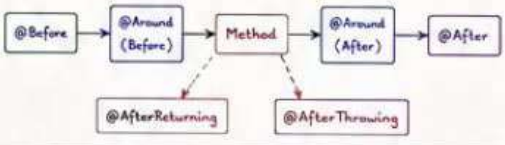
```
@Configuration
@EnableAspectJAutoProxy
public class AppConfig {
   @Bean
   public LoggingAspect loggingAspect() {
      return new LoggingAspect();
   }
}

@Aspect
public class LoggingAspect {
   @Pointcut("execution(* com.example.service.*.*(..))")
   public void allMethods() {}

   @Before("allMethods()")
   public void logBefore(JoinPoint jp) {
      System.out.println("Before: " + jp.getSignature());
   }
}
```

10. @Around Advice Example

- Has control over method execution.
- Can decide whether to proceed or not.
- Example :

```
@Around("allMethods()")
public Object logAround(ProceedingJoinPoint pjp) throws Throwable {
   System.out.println("Before Method");
   Object result = pjp.proceed(); // proceed to target method
   System.out.println("After Method");
   return result;
}
```

11. Order of Advice Execution



12. Introduction

- Allows adding new methods or fields to existing class.
- Implemented using @DeclareParents.
- Example :

```
public interface IsAuditable {
   void setCreatedBy(String user);
```

13. Proxy in Spring AOP

- Spring AOP uses proxy objects.
- Types :
  - JDK Dynamic Proxy (for interfaces)
  - CGLIB Proxy (for classes)
- Proxy intercepts method calls and applies advices.

14. Important Annotations

@Aspect	– Marks a class as Aspect
@Before	– Advice before method execution
@AfterReturning	– After successful execution
@AfterThrowing	– After exception is thrown
@After (finally)	– After execution
@Around	– Around method execution
@Pointcut	– Declares a pointcut
@EnableAspectJAutoProxy	– Enables Spring AOP
@DeclareParents	– Introduction

15. Use Cases (Cross-Cutting Concerns)

- ✓ Logging
- ✓ Transaction Management
- ✓ Security
- ✓ Caching
- ✓ Performance Monitoring
- ✓ Exception Handling
- ✓ Auditing

16. Full Example (Annotation Based)

```
@Service
public class UserService {
   public void saveUser() {
      System.out.println("User saved");
   }
}
@Aspect
public class LogAspect {
   @Pointcut("execution(* com.example.service.*.*(..))")
   public void allMethods() {}

   @Before("allMethods()")
   public void logBefore(JoinPoint jp) {
      System.out.println("Before: " + jp.getSignature());
   }

   @AfterReturning("allMethods()")
   public void logAfter() {
      System.out.println("After Returning");
   }

   @AfterThrowing("allMethods()")
   public void logException() {
      System.out.println("Exception Occurred");
   }

   @After("allMethods()")
   public void logFinally() {
      System.out.println("Finally");
   }
}
```

17. Best Practices

SPRING BOOT - COMPLETE SYLLABUS NOTES

Definition : Spring Boot is an extension of Spring Framework that makes it easy to create stand-alone, production-grade, Spring based applications with minimal configuration.

Benefits

- ✓ Auto-configuration
- ✓ Starter dependencies
- ✓ Embedded servers
- ✓ Production-ready features
- ✓ Minimal XML configuration
- ✓ Easy testing and deployment

1. Introduction to Spring Boot

- Spring Boot - Overview and Features
- Problems with Traditional Spring Configuration
- Spring Boot Philosophy - Convention over Configuration
- Standalone vs Traditional Spring Application
- Spring Boot Versions and Compatibility

2. Setting up Spring Boot Project

- Using Spring Initializr (start.spring.io)
- Project Metadata (Group, Artifact, Dependencies)
- Project Structure Explanation
- build.gradle and pom.xml overview
- Running Spring Boot Application

Spring Boot Project Structure

```
src
└── main
    ├── java
    │   └── com.example.demo
    │       └── DemoApplication.java
    └── resources
        ├── application.properties
        ├── static/
        └── templates/
└── test
    └── java
```

3. Spring Boot Starters

- What are Starters?
- Common Starters
  - spring-boot-starter-web
  - spring-boot-starter-data-jpa
  - spring-boot-starter-security
  - spring-boot-starter-test
  - spring-boot-starter-thymeleaf
- Custom Starters

4. Auto-Configuration

- What is Auto-Configuration?
- How it works? @EnableAutoConfiguration
- Condition Annotations (@Conditional...)
- Excluding Auto-Configuration

5. Application Properties / Configuration

- application.properties vs application.yml
- Externalized Configuration
- Property Sources
- Profiles (application-dev.properties)
- @Value, @ConfigurationProperties

Example - application.properties

```
server.port = 8080
spring.datasource.url = jdbc:mysql://localhost:3306/db
spring.datasource.username = root
spring.datasource.password = root
spring.jpa.hibernate.ddl-auto=update
```

6. Embedded Servers

- Embedded Tomcat (Default)
- Switching to Jetty / Undertow
- Configuring Server Properties (port, context-path, SSL)

7. Spring Boot Annotations

- @SpringBootApplication
  - @Configuration
  - @EnableAutoConfiguration
  - @ComponentScan
- @Configuration
- @Bean
- @Component, @Service, @Repository, @Controller
- @RestController

8. Creating RESTful Web Services

- @RestController and @RequestMapping
- Path Variables, Request Parameters
- Request Body and Response
- ResponseEntity and Status Codes
- Exception Handling with @ControllerAdvice

Example - REST Controller

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id){
        return userService.getUser(id);
    }
}
```

9. Data Access with Spring Boot

- Spring Data JPA with Spring Boot
- Configure DataSource
- Entity Mapping (@Entity, @Id, @Table)
- Repository Interfaces (JpaRepository)
- CRUD Operations
- Custom Queries (@Query)
- Pagination and Sorting

Spring Data Flow

Controller → Service → Repository → Database

10. Spring Boot Actuator

- What is Actuator?
- Adding Actuator dependency
- Monitoring Endpoints (health, info, metrics, beans, env, mappings)
- Custom Health Indicators

11. Spring Boot Security Basics

- Adding Spring Security
- Basic Authentication
- Form Login
- Roles and Authorities
- In-Memory Authentication
- Password Encoding
- Securing REST APIs

Example - InMemory User

```
@Bean
public UserDetailsService userDetailsService(){
    UserDetails user = User.withDefaultPasswordEncoder()
        .username("admin").password("admin");
        .roles("ADMIN").build();
    return new InMemoryUserDetailsManager(user);
}
```

12. Exception Handling

- Using @ControllerAdvice
- @ExceptionHandler
- Custom Exception Responses

13. Testing in Spring Boot

- Unit Testing with @SpringBootTest
- Testing Controllers (MockMvc)
- Testing Services and Repositories
- Test Slices (@WebMvcTest, @DataJpaTest)

14. Build and Run

- Running with Maven/Gradle
- Executable JAR / WAR
- java -jar application.jar
- Spring Boot DevTools (Automatic Restart)

15. Deployment

- Packaging as JAR/WAR
- Deploying on Tomcat / Any Server
- Docker Deployment (Basic)
- Cloud Deployment (Heroku / AWS / GCP)

16. Advanced Topics

- Logging (Logback Configuration)
- Spring Boot Profiles
- Custom Starters
- Scheduling Tasks (@Scheduled)
- Email Sending
- File Upload & Download
- Caching (Spring Cache Abstraction)
- Validation (@Valid)

17. Spring Boot vs Spring Framework

Feature	Spring Framework	Spring Boot
Configuration	Manual (XML/Java Config)	Auto-configuration
Server	External (Tomcat etc.)	Embedded Server
Dependencies	Manual	Starter Dependencies

18. Life Cycle of Spring Boot Application

1. Start Application (main method)
2. Initialize SpringApplication
3. Load Application Context

19. Important Files

- application.properties / application.yml
- logback-spring.xml (Logging)
- banner.txt (Custom Banner)
- data.sql / schema.sql (Database Initialization)

20. Best Practices

SPRING REST API – COMPLETE SYLLABUS NOTES

Rakesh Nikumbh

Definition: Spring REST is a module of Spring Framework that helps to create RESTful web services. It uses HTTP methods to perform CRUD operations.

Benefits
- ✓ Lightweight
- ✓ Uses standard HTTP methods
- ✓ Platform independent
- ✓ Easy integration with front-end
- ✓ Supports multiple data formats (JSON, XML, etc.)

1. Introduction to REST

- REST stands for Representational State Transfer.
- It is an architectural style for designing networked applications.
- Uses stateless communication.
- Resources are identified using URIs.
- Uses standard HTTP methods.

2. REST Principles

- Client-Server
- Stateless
- Cacheable
- Uniform Interface
- Layered System
- Code on Demand (optional)

Uniform Interface includes:
- ✓ Resource identification in requests
- ✓ Manipulation of resources through representations
- ✓ Self-descriptive messages
- ✓ Hypermedia as the engine of application state

3. REST vs SOAP

	REST	SOAP
Protocol	HTTP	HTTP, SMTP, etc.
Data Format	JSON, XML, Text, etc.	XML Only
Message	Lightweight	Heavyweight
State	Stateless	Can be Stateful
Performance	Faster	Slower
Use Case	Web, Mobile, Microservices	Enterprise Applications

4. Setting up REST API in Spring

- Add dependencies
  - spring-web
  - (optional) spring-boot-starter-web
- Create Spring Boot / Spring MVC project
- Enable component scanning
- Create REST Controller

5. Annotations in Spring REST

- @RestController – Combines @Controller and @ResponseBody
- @RequestMapping – Map HTTP requests
- @GetMapping – Handle GET requests
- @PostMapping – Handle POST requests
- @PutMapping – Handle PUT requests
- @DeleteMapping – Handle DELETE requests
- @PathVariable – Get value from URI
- @RequestParam – Get query parameter
- @RequestBody – Map HTTP request body to Java object
- @ResponseStatus – Set HTTP status code
- @CrossOrigin – Enable CORS

6. HTTP Methods

Method	Description
GET	Retrieve resource
POST	Create new resource
PUT	Update entire resource
PATCH	Update partial resource
DELETE	Delete resource

7. Request Mapping

- Class Level Mapping

```
@RestController
@RequestMapping("/api/users")
public class UserController { ... }
```

- Method Level Mapping

```
@GetMapping("/{id}")
@PostMapping
@PutMapping("/{id}")
@DeleteMapping("/{id}")
```

8. Path Variables and Request Parameters

- Path Variable Example

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
    // code
}
```

- Request Parameter Example

```
@GetMapping("/users")
public List<User> getUsers(@RequestParam
    (required = false) String name) {
    // code
}
```

9. Request Body and Response Body

- Request Body

```
@PostMapping("/users")
public User createUser(@RequestBody User user) {
   // code
}
```

- Response Body

```
@GetMapping("/{id}")
public @ResponseBody User getUser(@PathVariable
  Long id) {
    // code
}
```

10. ResponseEntity

- Used to customize the whole HTTP response.
- Contains body, headers and status code.

```
@GetMapping("/{id}")
public ResponseEntity<User> getUser(@PathVariable
    Long id) {
    User user = service.getUser(id);
    return new ResponseEntity<>(user, HttpStatus.OK);
}
```

11. Status Codes

- 200 OK – Success
- 201 Created – Resource created
- 204 No Content – Success, no content
- 400 Bad Request – Invalid request
- 401 Unauthorized – Authentication required
- 403 Forbidden – Access denied
- 404 Not Found – Resource not found
- 500 Internal Server Error – Server error

12. Data Formats

- JSON – Most used format
- XML – Supported
- Other – Plain Text, YAML, etc.
- Use @RequestBody and @ResponseBody with Jackson (JSON) or JAXB (XML)

13. Exception Handling

- Use @ExceptionHandler
- Use @ControllerAdvice for global handling

```
@ControllerAdvice
public class GlobalExceptionHandler {
   @ExceptionHandler(ResourceNotFoundException.class)
   public ResponseEntity<String> handleNotFound() {
      return new ResponseEntity<>("Resource Not Found",
                          HttpStatus.NOT_FOUND);
   }
}
```

14. Validation

- Use Hibernate Validator
- Add dependency: spring-boot-starter-validation
- Use annotations like @NotNull, @Size, @Email

```
public class User {
   @NotNull
   private String name;
   @Email
   private String email;
}
```

15. CORS (Cross Origin Resource Sharing)

- Enables access from different domains.
- Use @CrossOrigin or WebMvcConfigurer.

```
@CrossOrigin(origins = "http://localhost:4200")
@RestController
public class UserController { ... }
```

16. Testing REST APIs

- Tools: Postman, Insomnia, Curl
- Steps:
  1. Create Request (URL + Method)
  2. Set Headers
  3. Send Request
  4. Check Response (Status + Body)

17. Example: CRUD REST API Endpoints

18. Best Practices

SPRING SECURITY - COMPLETE SYLLABUS NOTES

Definition : Spring Security is a powerful and highly customizable authentication and access-control framework for Spring based applications. It provides protection against common attacks and enables secure enterprise applications.

Benefits

- ✓ Strong authentication & authorization
- ✓ Protection against common attacks (CSRF, XSS, Session Fixation, etc.)
- ✓ Fine grained access control
- ✓ Easy integration with Spring ecosystem
- ✓ Supports form login, OAuth2, JWT, LDAP, etc.

1. Introduction to Spring Security

- Part of Spring Framework.
- Provides authentication, authorization and protection against attacks.
- Highly customizable and extensible.
- Integrates with Spring Boot seamlessly.

2. Core Concepts

- Authentication - Who are you?
- Authorization - What are you allowed to do?
- Principal - Authenticated user.
- GrantedAuthority - Roles/Permissions.
- UserDetails - Core user information.
- SecurityContext - Holds authentication information.
- FilterChainProxy - Secures incoming requests.

3. Architecture

4. Setting up Spring Security

- Add dependency : spring-boot-starter-security
- Auto-configuration by Spring Boot
- Default user is generated if no user is defined.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

5. Security Configuration

- Create SecurityConfig class.
- Extend WebSecurityConfigurerAdapter (in older versions) or define SecurityFilterChain bean (latest approach).

Example (Spring Boot 3+) :

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
      throws Exception {
    http
      .authorizeHttpRequests(auth -> auth
        .requestMatchers("/", "/login", "/css/**").permitAll()
        .anyRequest().authenticated())
      .formLogin(form -> form.loginPage("/login").permitAll())
      .logout(logout -> logout.logoutSuccessUrl("/login?logout"))
```

6. Authentication

Spring Security supports multiple authentication mechanisms.

- Form Login (Default)
- Basic Authentication
- Custom Login (UsernamePasswordAuthenticationFilter)
- Remember Me Authentication
- Social Login (OAuth2)

7. UserDetailsService

- Loads user-specific data.
- Core method : loadUserByUsername(username)
- Can be implemented using DB, LDAP, In-Memory etc.

Example :

```
@Service
public class CustomUserDetailsService implements UserDetailsService {
  @Autowired
  private UserRepository userRepository;

  @Override
  public UserDetails loadUserByUsername(String username)
      throws UsernameNotFoundException {
    User user = userRepository.findByUsername(username);
    if(user == null)
      throw new UsernameNotFoundException("User not found");
    return new org.springframework.security.core.userdetails.
        User(user.getUsername(), user.getPassword(),
            user.getAuthorities());
  }
}
```

8. Password Encoding

- Passwords must be encoded.
- Use PasswordEncoder (BCryptPasswordEncoder recommended).

Example :

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

9. Authorization (Access Control)

- Role Based Access Control (RBAC)
- Method level security
- URL based access control
- Expressions

Example (URL based) :

```
http.authorizeHttpRequests(auth -> auth
  .requestMatchers("/admin/**").hasRole("ADMIN")
  .requestMatchers("/user/**").hasAnyRole("USER","ADMIN")
  .anyRequest().authenticated())
```

Example (Method level) :

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser() { ... }
```

10. Roles and Authorities

- Role - High level permissions (ADMIN, USER)
- Authority - Fine grained permissions (READ, WRITE)
- In Spring Security, roles are prefixed with ROLE_

11. CSRF Protection

- CSRF (Cross Site Request Forgery) protection enabled by default.
- Token is added to forms and validated.
- Disable for stateless REST APIs.

```
http.csrf(csrf -> csrf.disable()); // for APIs
```

12. Logout Handling

- Invalidate session and clear authentication.
- Redirect to login page after logout.

Example :

```
http.logout(logout -> logout
      .logoutUrl("/logout")
      .logoutSuccessUrl("/login?logout")
      .invalidateHttpSession(true)
      .deleteCookies("JSESSIONID"));
```

13. Remember Me Authentication

- Used to remember user across sessions.
- Uses cookies.
- Requires token repository.

Example :

```
http.rememberMe(remember -> remember
    .key("mySecretKey")
    .tokenValiditySeconds(86400));
```

14. Exception Handling

- Handle authentication & authorization exceptions.
- Custom pages for Access Denied and Login.

Example :

```
http.exceptionHandling(ex -> ex
  .accessDeniedPage("/access-denied"))
```

15. Method Level Security

- Enable with @EnableMethodSecurity
- Annotations :
  - @PreAuthorize
  - @PostAuthorize
  - @Secured
  - @RolesAllowed

Example :

```
@PreAuthorize("hasRole('ADMIN')")
public String adminDashboard() { ... }
```

16. Integrations

- OAuth2 / OpenID Connect
- JWT (JSON Web Token)
- LDAP Authentication
- SAML
- Social Login (Google, GitHub, Facebook etc.)

17. Important Components

SPRING JWT - COMPLETE SYLLABUS NOTES

Rakesh Nikumbh

Definition : JWT (JSON Web Token) is a compact and self-contained way for securely transmitting information between parties as a JSON object. In Spring, JWT is commonly used for Stateless Authentication in RESTful APIs.

Benefits

- ✓ Stateless Authentication
- ✓ Secure and Compact
- ✓ Can be used across different domains
- ✓ Scalable and Performance friendly
- ✓ Supports Authorization (Roles/Permissions)

1. Introduction to JWT

- JWT is an open standard (RFC 7519).
- It represents claims between two parties.
- Mainly used for Authentication & Information Exchange.
- Used in Stateless architecture.
- Common in Microservices & REST APIs.

2. JWT Structure

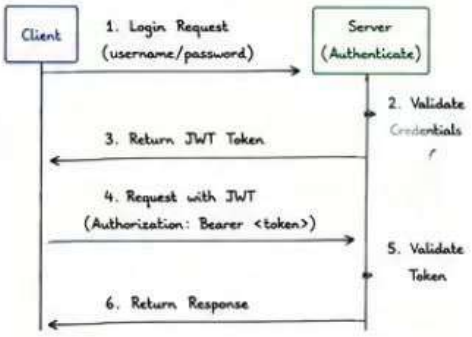
Header (Base64Url Encoded) . Payload (Base64Url Encoded) . Signature (Base64Url Encoded)

Header.Payload.Signature

- Header : Algorithm, Type of token
- Payload : Claims (user details, roles, exp, etc.)
- Signature : Used to verify the token

Header + Payload + Secret Key (or Public Key)

3. JWT Workflow



4. Dependencies (Maven)

```
<dependencies>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.11.5</version>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.11.5</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.11.5</version>
        <scope>runtime</scope>
</dependencies>
```

5. JWT Utility Class (Token Generation)

```
public String generateToken(String username, List<String> roles){
  return Jwts.builder()
    .setSubject(username)
    .claim("roles", roles)
    .setIssuedAt(new Date())
    .setExpiration(new Date(System.currentTimeMillis() + 1000*60*60)) // 1h
```

6. Claims in JWT (Payload)

Claim	Description
iss	Issuer of the token
sub	Subject (usually username)
aud	Auidence
exp	Expiration time (timestamp)
nbf	Not before (validity start time)
iat	Issued at (timestamp)
jti	JWT ID (unique identifier)
roles	Custom claim for user roles

7. Token Validation (Parsing)

```
public Claims validateToken(String token){
    return Jwts.parserBuilder()
        .setSigningKey(secretKey)
        .build()
        .parseClaimsJws(token)
        .getBody();
}
```

8. Extract Information from Token

```
Claims claims = validateToken(token);
String username = claims.getSubject();
List<String> roles = claims.get("roles", List.class);
Date expiry = claims.getExpiration();
```

9. JWT Authentication Filter

- Intercepts every request.
- Extracts token from Authorization header.
- Validates the token.
- If valid, set Authentication in SecurityContext.

```
String token = header.substring(7); // Bearer <token>
if(jwtUtil.validateToken(token)){
    String username = jwtUtil.getUsername(token);
    List<String> roles = jwtUtil.getRoles(token);
    UsernamePasswordAuthenticationToken auth =
        new UsernamePasswordAuthenticationToken(
            username, null, authorities);
    SecurityContextHolder.getContext().setAuthentication(auth);
}
```

10. Spring Security Configuration

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
return http
  .csrf(AbstractHttpConfigurer::disable)
  .sessionManagement(session -> session.sessionCreationPolicy(
    SessionCreationPolicy.STATELESS))
  .authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()
    .anyRequest().authenticated())
  .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
  .build();
}
```

11. Authentication Endpoints

12. Storing JWT on Client

- LocalStorage (Web Applications)
- Secure Storage / SharedPreferences (Mobile)
- HttpOnly Cookie (More Secure for Web)

13. Sending JWT with Request

- Add token in Authorization header
- Format : Bearer <token>

```
Authorization: Bearer eyJhbGciOiJIUzUxMIJ9....
```

14. Token Expiration & Refresh

- Set short expiry time for access token (e.g. 15m).
- Provide refresh token with longer expiry (e.g. 7d).
- Use refresh token to get a new access token.

Access Token (Short lived) → Refresh Token → New Access Token

15. Blacklisting / Logout

- Since JWT is stateless, logout is not automatic.
- Add token to blacklist in DB/Redis.
- Check blacklist in filter during validation.

```
if (blacklistService.isBlacklisted(token)){
    throw new RuntimeException("Token is blacklisted");
}
```

16. Best Practices

- ✓ Use strong secret key (256-bit or more)
- ✓ Keep token size small
- ✓ Set appropriate expiration time
- ✓ Never store sensitive data in payload
- ✓ Use HTTPS always
- ✓ Implement refresh token mechanism
- ✓ Handle exceptions properly

17. Use Cases

- User Authentication
- Stateless REST APIs
- Microservices Security
- Single Sign-On (SSO)
- Mobile & SPA Applications

18. Example Flow Summary

1. User logs in with credentials.
2. Server validates and returns JWT.
3. Client stores JWT and sends with each request.
4. Server validates JWT and returns response.
5. If token expired → use refresh token.

CHECKLIST

☆ SPRING MICROSERVICES – COMPLETE SYLLABUS NOTES ☆

Definition : Microservices architecture is an approach to develop a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. Each service is built around business capabilities and independently deployable.

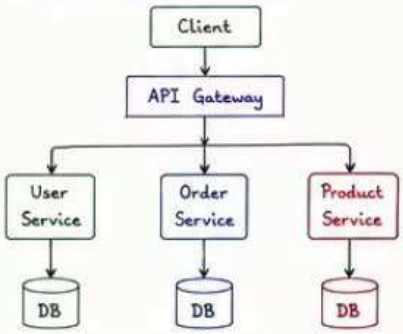
Benefits

- ✓ Independent Development & Deployment
- ✓ Scalability – Scale only what you need
- ✓ Technology Diversity
- ✓ Fault Isolation – Failure in one service doesn't affect others
- ✓ Continuous Delivery & Faster Releases
- ✓ Easy Maintenance & High Availability

1. Introduction to Microservices

- What is Monolithic Architecture?
- Problems with Monolith
- What are Microservices?
- Characteristics of Microservices
  - Small, Independent, Loosely Coupled
  - Single Responsibility
  - Decentralized Data Management
  - Independent Deployment
  - Fault Isolation

2. Microservices Architecture



3. Setting up Environment

- Install JDK, IDE, Maven/Gradle
- Spring Boot Dependency
- Project Structure for Services
- Common Properties (application.yml)

```yaml
spring:
  application:
    name: user-service
  server:
    port: 8081
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
```

4. Creating First Microservice

- Using Spring Initializr
- Build a Simple REST API
- Run and Test the Service

```java
@RestController
public class UserController {
  @GetMapping ("/users/{id}")
  public User getUser(@PathVariable Long id){
     return new User(id, "John");
  }
}
```

5. Service Discovery with Eureka

- What is Service Discovery?
- Eureka Server Setup
- Registering Microservices with Eureka
- High Availability for Eureka

6. API Gateway with Spring Cloud Gateway

- What is API Gateway?
- Benefits of API Gateway
- Route Configuration
- Predicates and Filters
- Centralized Authentication & Rate Limiting

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://USER-SERVICE
          predicates:
            - Path=/api/users/**
        - id: order-service
          uri: lb://ORDER-SERVICE
          predicates:
            - Path=/api/orders/**
```

7. Inter-Service Communication

- Approaches
  - RestTemplate (Synchronous)
  - WebClient (Reactive)
  - Feign Client (Declarative)
- Example using Feign Client

```java
@FeignClient(name = "order-service")
public interface OrderClient {
  @GetMapping ("/orders/{id}")
    Order getOrder(@PathVariable Long id);
}
```

8. Externalized Configuration

- What is Config Server?
- Centralized Configuration Management
- Git based Configuration
- Refreshing Configuration

```yaml
spring:
  cloud:
    config:
      uri: http://localhost:8888
      name: user-service
      profile: dev
```

9. Fault Tolerance with Resilience4j

- What is Circuit Breaker?
- Fallback Methods
- Retry, RateLimiter, TimeLimiter, Bulkhead
- Example – Circuit Breaker

```java
@CircuitBreaker(name = "orderService",
        fallbackMethod = "fallbackMethod")
public Order getOrder(Long id){
        // call external service
}
public Order fallbackMethod(Long id, Exception ex){
   return new Order(id, "FAILED");
}
```

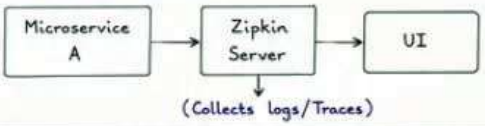
10. Load Balancing with Spring Cloud LoadBalancer

- Client Side Load Balancing
- Integration with Eureka
- Ribbon is deprecated – Use LoadBalancer

```java
@LoadBalanced
@Bean
public WebClient.Builder webClientBuilder(){
    return WebClient.builder();
}
```

11. Distributed Logging with ELK / Sleuth

- What is Distributed Logging?
- Using Spring Cloud Sleuth + Zipkin
- Trace ID and Span ID
- Integration Flow

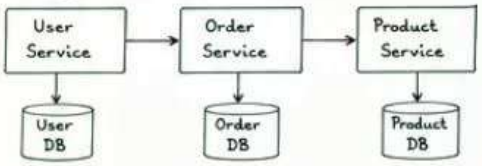


12. Messaging with RabbitMQ / Kafka

- Why Messaging?
- RabbitMQ / Kafka Basics
- Producer – Consumer Example
- Asynchronous Communication

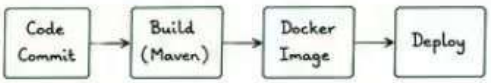
13. Database per Service

- Each Microservice has its own Database
- No Shared Database
- Polyglot Persistence



14. Deployment

- Build Executable JAR
- Dockerize Microservices
- Docker Compose / Kubernetes
- CI/CD Pipeline (Jenkins/GitHub Actions)



16. Security in Microservices

- Centralized Authentication (OAuth2 / JWT)
- API Gateway Security
- Propagating JWT in Request

17. Important Concepts

- Stateless Services
- Immutable Infrastructure

Best Practices

- ✓ Keep services small & focused
- ✓ Use proper naming conventions

SPRING AI - COMPLETE SYLLABUS NOTES

Rakesh Nikumbh

Definition : Spring AI is a framework that simplifies the integration of Artificial Intelligence capabilities into Spring Boot applications. It provides abstractions and starters for working with different AI models, providers and techniques.

Benefits
- ✓ Simplifies AI model integration
- ✓ Support for Multiple AI Providers
- ✓ Consistent Abstraction Layer
- ✓ Easy to switch between models
- ✓ Build AI-Powered features quickly
- ✓ Works seamlessly with Spring ecosystem

1. Introduction to Spring AI
- New project under Spring ecosystem.
- Provides abstraction for AI models.
- Supports LLMs, Embeddings, Vector Stores, Chat, Image, and more.
- Works with multiple AI providers.
- Designed for Enterprise Applications.

2. Spring AI Features
- Model Abstraction
- Prompt Management
- Chat & Completion
- Embeddings & Vector Store
- Function Calling (Tools)
- Retrieval Augmented Generation (RAG)
- Streaming Responses
- Multi-Model Support
- Observability & Monitoring

3. Supported AI Models (Examples)
- OpenAI (GPT-3.5, GPT-4, embeddings)
- Azure OpenAI
- Google Gemini
- Hugging Face Models (Local & Hosted)
- Ollama (Local LLMs)
- Cohere, Mistral AI and more...

4. Project Setup
Dependencies (Maven)

```
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-openai</artifactId>
</dependency>
```

Optional (for Vector Store with PGVector)

```
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-vector-store-pgvector</artifactId>
</dependency>
```

application.yml

```
spring:
  ai:
    openai:
      api-key: YOUR_OPENAI_API_KEY
      chat:
        options:
          model: gpt-3.5-turbo
          temperature: 0.7
```

5. Model Abstraction
- Spring AI provides Model interface.
- Different models can be used through same interface.

Example :

```
@Autowired
private ChatModel chatModel;
String response = chatModel.call(
```

6. Chat & Completion
- Perform chat conversation with LLM.
- Support for single turn & multi-turn chat.

Example :

```
ChatResponse response = chatModel.call(
    new Prompt("Hello, how can you help me?"));
System.out.println(response.getResult().getOutput().getText());
```

7. Prompt Management
- Manage prompts using PromptTemplate.
- Inject dynamic values in prompts.

Example :

```
PromptTemplate template = new PromptTemplate(
    "Summarize the following text:\n{text}");
Prompt prompt = template.create(Map.of("text", content));
String result = chatModel.call(prompt).getResult().getOutput().getText();
```

8. Embeddings
- Convert text into vector embeddings.
- Useful for similarity search & RAG.

Example :

```
@Autowired
private EmbeddingModel embeddingModel;
List<Double> vector = embeddingModel.embed(
                    "Spring AI is amazing!");
```

9. Vector Store Integration
- Store and search embeddings in Vector DB.
- Supported Vector Stores : PGVector, Chroma, Pinecone, Weaviate, Redis and more.

Example (Add Document)

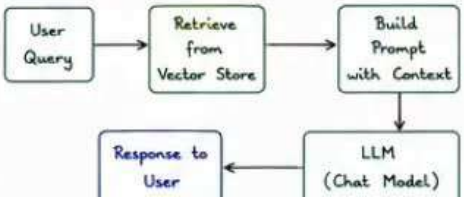
```
VectorStore vectorStore = ...;
Document doc = new Document("doc1", "Spring AI
  helps build AI apps easily", Map.of("type","note"));
vectorStore.add(List.of(doc));
```

Example (Similarity Search)

```
SearchRequest request = SearchRequest.builder()
    .query("What is Spring AI?")
    .topK(3).build();
List<Document> results = vectorStore.similaritySearch(request);
```

10. Retrieval Augmented Generation (RAG)
- Combine LLM with your own data.
- Retrieve relevant docs from Vector Store and send to LLM.

Flow :



11. Function Calling (Tools)
- Allow LLM to call external functions.
- Useful for real-world integrations.

Example :

```
@Tool
public String getWeather(@Param("city") String city){
  // call external API
  return "Weather in " + city + " is sunny";
}
```

12. Streaming Responses
- Get partial responses as they are generated.
- Useful for better user experience.

Example :

```
ChatOptions options = ChatOptions.builder()
    .withStreaming(true).build();
chatModel.stream(prompt, options)
    .forEach(chunk -> System.out.print(chunk.getText()));
```

13. Multi-Model Support
- Easily switch between different AI providers using configuration.
- Compare results from multiple models.

14. Observability & Monitoring
- Integrates with Micrometer, Tracing.
- Monitor token usage, latency, errors.
- Works with Spring Boot Actuator.

15. Advanced Topics
- ✓ Chat Memory (Conversation History)
- ✓ System Prompts
- ✓ Guardrails & Content Filtering
- ✓ Tool Calling with Multiple Functions
- ✓ Fine-tuning & Custom Models
- ✓ Batch Processing
- ✓ Rate Limiting & Caching
- ✓ Error Handling & Fallbacks
- ✓ Cost Monitoring
- ✓ Security (API Key Management)

16. Use Cases
- AI Chatbots
- Document Q&A
- Code Generation & Explanation
- Summarization
- Recommendation Systems
- Data Extraction
- AI Agents with Tools
- Intelligent Search

17. Best Practices
- ✓ Keep prompts clear and concise
- ✓ Manage conversation memory wisely